BE-03: Technology Selection

Intelligent Kanban System with an AI Assistant
for Managing Tasks by Voice and Text

Symbat Saparbay | Akbope Bakytkeldy

YouTrack: DEMO-22

Contents

1 Goal **2**

2 Requirements Driving the Choice **2**

3 Architecture **3**

3.1 AI Pipeline 3

4 Speech-to-Text **3**

4.1 Comparison 3

4.2 Decision: Self-Hosted STT with Language Routing and a Cloud Fallback 4

5 LLM for Command Understanding **4**

5.1 What the Model Must Do 4

5.2 Comparison 5

5.3 Decision: Hybrid Behind a Single `CommandParser` Interface 5

5.4 Protection Against Invented Data 6

6 Frontend **6**

7 Backend **6**

8 Database and Storage **7**

9 Security and Data Protection **7**

10 Notifications **8**

11 Final Stack **8**

12 Validating the Choice **9**

13 Risks **9**

14 References **9**

1 Goal

Select and justify the technologies for a **commercial Kanban product** used by company employees, project managers, teachers and students. The AI assistant is built into the workflow: the user speaks or types a command, the system understands it, shows the proposed action and performs it on the board after confirmation.

2 Requirements Driving the Choice

ID	Requirement	Impact on the stack
R1	Kazakh, Russian and English, including mixed speech (Kazakh + Russian + English technical terms)	STT and LLM quality on Kazakh is the main criterion
R2	Eight AI actions: Create Task, Update Task, Change Status, Assign User, Change Deadline, Change Priority, Search Tasks, Project Summary	LLM with reliable structured output / tool calling
R3	AI never changes data without confirmation or when a command is ambiguous; never invents assignees, deadlines or tasks	Deterministic validation in the backend; the AI only proposes
R4	Complete product: authentication, workspaces, projects, members, tasks, comments, history, notifications	Relational database, RBAC, real-time updates
R5	Response to a voice command within a few seconds	Fast STT, asynchronous processing
R6	Protection of user data and voice recordings	Encryption, limited retention, access control
R7	Scalability and room for growth	Stateless API, separate AI worker, models swappable by configuration
R8	Reliability: AI errors, API outages, repeated requests	Fallback providers, idempotency, queues
R9	Visible development trail (GitLab, YouTrack)	GitLab CI, tests, history

3 Architecture

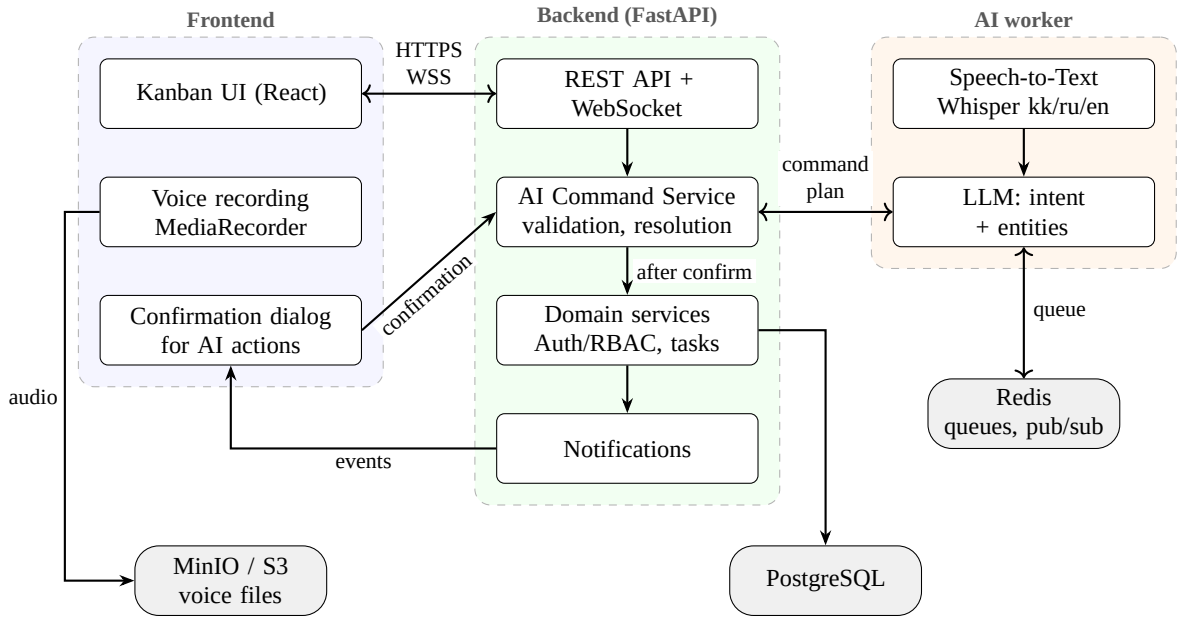


Figure 1: Component architecture. The AI worker reads audio from MinIO and returns an action plan; only the backend changes data, and only after confirmation.

3.1 AI Pipeline

Voice Input → Speech-to-Text → Intent Recognition → Entity Extraction →
Validation & Resolution → User Confirmation → Backend Action → Kanban Update

Step	Where	Technology
Voice Input	Browser	MediaRecorder API (WebM/Opus), uploaded to MinIO via the backend
Speech-to-Text	AI worker	Whisper large-v3-turbo + Whisper fine-tuned on Kazakh (Section 4)
Intent Recognition + Entity Extraction	AI worker	LLM with tool calling: one call returns a list of actions with parameters (Section 5)
Validation & Resolution	Backend, no LLM	Pydantic schemas, assignee lookup among project members, task search (pg_trgm), date and permission checks
User Confirmation	Frontend	Preview cards with missing and ambiguous fields highlighted; transcript and fields can be edited
Backend Action	Backend	The same domain services as manual UI actions (shared permission checks and audit)
Kanban Update	Frontend	WebSocket event; the board updates for all members

Key principle. The LLM only *understands the command and proposes* an action. Everything that changes data is ordinary backend code that runs after the user confirms, with the same permission checks as manual actions. As a result, the AI cannot change someone else's task or create a task without confirmation.

4 Speech-to-Text

4.1 Comparison

Criterion	Whisper large-v3-turbo (self-hosted)	Whisper fine-tuned on KSC2 (self-hosted)	ElevenLabs Scribe (cloud)	OpenAI gpt-4o-transcribe (cloud)
Kazakh	Weak: confuses it with Turkish; WER ≈77% in a published study	**Good: WER ≈15–18%** on the KSC2 test set	Good (vendor-reported WER 3.1% on FLEURS)	Moderate to good
Russian / English	**Excellent**	Worse than the base model	Excellent	Excellent
Mixed speech	Automatic detection	Partial	Automatic detection	Automatic detection
Speed	≈1–2 s per 10 s of audio (GPU)	Similar	1–3 s + network	1–3 s + network
Cost	Hardware only	Hardware only	≈$0.22 / hour	$0.006 / min
Privacy	**Voice stays on our server**	**Voice stays on our server**	Data sent to the vendor	Data sent to the vendor
Scaling	GPU workers	GPU workers	Managed	Managed

4.2 Decision: Self-Hosted STT with Language Routing and a Cloud Fallback

1. **Automatic language detection.** Whisper large-v3-turbo (`faster-whisper`) computes language probabilities, which are restricted to Kazakh, Russian and English and renormalised.
2. **Routing without a hint.** The Kazakh model is used if Kazakh is the most likely language *or* $p(\text{kk}) \geq 0.30$. The threshold handles mixed speech: a Kazakh sentence with Russian and English terms is often detected as Russian, but with a noticeable Kazakh share. Otherwise, Russian and English speech is transcribed by Whisper large-v3-turbo with the detected language.
3. **Language selected by the user.** If the user selects a language in the UI, detection is skipped: Kazakh goes to the Kazakh model, Russian and English to Whisper large-v3-turbo. For mixed speech, the UI suggests Kazakh.
4. The user can **correct the transcript** before it is processed.
5. **Fallback:** if the local STT is unavailable or overloaded, a cloud provider is used (enabled in the workspace settings).

The 0.30 threshold is a starting value; it is a parameter tuned with the benchmark. The benchmark (`benchmarks/stt_benchmark.py`) evaluates multilingual models separately in two modes: **auto** (automatic language detection, the main product scenario) and **hint** (language selected by the user). For auto mode it also reports language-detection accuracy and how mixed recordings are routed. The final Kazakh model is chosen from the benchmark results.

Rationale. Voice is the most sensitive data in the system. Self-hosted STT addresses privacy and keeps costs flat as the number of users grows, while the fine-tuned model compensates for Whisper's weakness in Kazakh. The cloud fallback improves reliability (R8).

5 LLM for Command Understanding

5.1 What the Model Must Do

From a single utterance, often with recognition errors and mixed languages, the model must produce an **action plan** that strictly follows a schema. Example for the command *«Создай задачу для Данияра: подготовить презентацию проекта до пятницы, высокий приоритет»* ("Create a task for Daniyar: prepare the project presentation by Friday, high priority"):

```
{
  "actions": [
    {
      "type": "create_task",
      "params": {
        "title": "Подготовить презентацию проекта",
        "assignee_mention": "Данияр",
        "deadline": "2026-10-23",
        "deadline_phrase": "до пятницы",
        "priority": "high"
      },
      "missing": [],
      "confidence": 0.92
    }
  ],
  "clarification_needed": null
}
```

Requirements: tool calling or JSON Schema, understanding of Kazakh, Russian and English, several assignments in one utterance, Kazakh case forms of names («Данияpға» → «Данияр»), and relative dates. Task titles are kept in the original language.

5.2 Comparison

Criterion	Local 7–8B LLM (Qwen 3 / ISSAI KazLLM via Ollama)	Cloud Flash/mini-class LLM (Gemini Flash, OpenAI mini)
Kazakh	Qwen: moderate; KazLLM: good, but weaker at tool calling	Good
Tool calling, several actions in one utterance	Unstable at 7–8B, especially in Kazakh	**Reliable**
Speed	3–15 s without a powerful GPU	**1–3 s**
Cost	Hardware only	Estimate: ≈1.5k input + 300 output tokens per command ≈ **$0.005 per command**
Scaling	Requires more GPUs	Managed
Privacy	Full	Only the command text and project member names are sent; no audio and no task contents

5.3 Decision: Hybrid Behind a Single `CommandParser` Interface

- **Default mode: a cloud Flash/mini-class LLM** with tool calling. For a real product, the deciding factors are reliable parsing of complex Kazakh commands, response time (the user is waiting for the card preview) and scaling without buying GPUs.
- **Private (on-premise) mode: a local LLM via Ollama** (Qwen 3 or KazLLM 8B). Enabled by configuration for organisations that cannot send text to external services; also used as a fallback when the cloud API is unavailable.
- Both providers implement the same interface and JSON schema. The specific model is chosen by testing on our 30+ scenarios: intent accuracy, field accuracy, rate of invented values and response time.

Why the version 2 decision (fully local) was changed. Without a powerful GPU, a local 7–8B model responds too slowly and parses several actions in one Kazakh utterance unreliably. This conflicts with the speed and AI-integration-quality criteria. Voice is still processed only on our server, and the local LLM mode remains available as an option.

5.4 Protection Against Invented Data

Risk	Mechanism
Invented assignee	The LLM returns only a mention (`assignee_mention`). The backend looks it up among project members, normalising Kazakh case endings, handling transliteration and using fuzzy matching. If there is no match or several matches, the field is flagged and the user picks from a list
Invented deadline	The LLM returns both the date and the source phrase (`deadline_phrase`). If the phrase is not in the transcript, the date is discarded. The backend checks that the date is not in the past, using the user's time zone
Wrong task when updating	The LLM returns a description of the task. The backend searches for candidates (pg_trgm on titles in accessible projects): none means "not found", more than one means a clarification question
Invented tasks in answers (Search, Summary)	The backend runs the query on real data. The LLM only builds the filter (status, assignee, deadline); the answer contains real rows from the database
Action without confirmation	Changes are applied only via `POST /ai-commands/{id}/confirm`

6 Frontend

Criterion	React + TypeScript (Vite)	Vue 3	Angular
Kanban drag-and-drop	dnd-kit, many ready-made components	vuedraggable	CDK Drag&Drop
Server state and real time	TanStack Query + WebSocket	Pinia + composables	RxJS
Shared types with the backend	Types generated from OpenAPI	Same	Same
Learning curve and adoption	Most widely used	Easy	Steep

Decision: React + TypeScript + Vite. Tailwind CSS and shadcn/ui for components, dnd-kit for dragging cards, TanStack Query for data fetching, an API client generated from the FastAPI OpenAPI specification (shared types between frontend and backend), and react-i18next for Kazakh, Russian and English. Voice is recorded with the browser MediaRecorder API, so no separate mobile app is needed.

7 Backend

Criterion	FastAPI	Django + DRF	Flask
Async (AI calls, WebSocket)	Native	Partial	Via extensions

Criterion	FastAPI	Django + DRF	Flask
Validation	Pydantic: the same schemas for AI actions, API and validation	Serializers	Manual
OpenAPI specification	Automatic	Via add-ons	Via add-ons
WebSocket	Built in	Django Channels	Via extensions

Decision: Python 3.12 + FastAPI, SQLAlchemy 2.0 (async) and Alembic. Modules are organised by domain: `auth`, `workspaces`, `projects`, `tasks`, `comments`, `notifications`, `ai_commands`, `audit`.

The **AI worker** is a separate process (ARQ or Celery + Redis). STT models are loaded into memory once, the API is never blocked, and workers scale independently.

8 Database and Storage

Criterion	PostgreSQL	MongoDB
Data model (workspace → project → board → column → task, members, roles)	Relational, foreign keys, transactions	Relations handled manually
Access control and integrity	Constraints at database level	In application code
Finding a task from a spoken description	pg_trgm; pgvector if needed	Atlas Search
Flexible AI data (transcript, plan)	JSONB	Native
Audit and history	Append-only table	Collection

Decision:

- **PostgreSQL 16**: main database (pg_trgm, optionally pgvector).
- **Redis**: AI job queue, pub/sub for WebSocket across API instances, rate limiting, idempotency keys.
- **MinIO** (S3-compatible storage): voice messages; replaced by S3 in the cloud without code changes.

Key entities (detailed in the Database Schema): `users`, `workspaces`, `workspace_members`, `projects`, `project_members`, `boards`, `columns`, `tasks`, `task_comments`, `task_history`, `notifications`, `voice_messages`, `ai_commands`, `audit_log`.

9 Security and Data Protection

Requirement	Solution
Authentication	JWT: short-lived access token and refresh token in an httpOnly cookie; passwords hashed with Argon2
Access control	RBAC: workspace roles (owner, admin, member) and project roles (manager, member, viewer); checked in every service, including AI actions
Voice messages	Private bucket, encryption at rest, access via short-lived presigned URLs, automatic deletion after 30 days (configurable); the transcript is kept in history

Requirement	Solution
Data sent to the cloud LLM	Only the command text and project member names; no audio, email addresses or contents of other tasks. On-premise deployments use the local LLM
Duplicate task creation	AI command status `pending` → `confirmed` → `executed`; confirmation is idempotent, so a repeated click creates nothing; warning about a similar recently created task
AI unavailability	Timeouts and retries, switch to the fallback provider, a clear message to the user; manual task creation always works
Action history	`task_history` (what, who, when, via UI or AI) and `audit_log`
Transport	HTTPS/WSS, CORS, rate limiting on AI endpoints

10 Notifications

- **Real-time in-app:** WebSocket for assignments, status changes, comments and mentions.
- **Email:** SMTP (Mailpit during development) for assignments and upcoming deadlines.
- **Deadline reminders:** a periodic job (ARQ cron or Celery beat) runs every 15 minutes and finds tasks due within 24 hours and overdue tasks.
- Users choose which notifications they receive.

11 Final Stack

Layer	Technology	Key reason
Languages	Kazakh, Russian, English, mixed speech	Product requirement
Frontend	React + TypeScript + Vite, Tailwind + shadcn/ui, dnd-kit, TanStack Query, react-i18next	Mature Kanban ecosystem, types from OpenAPI
Voice recording	MediaRecorder API (browser)	No separate app needed
Backend	Python 3.12 + FastAPI, SQLAlchemy 2.0, Alembic	Async, Pydantic, automatic OpenAPI
AI worker	ARQ / Celery + Redis	Does not block the API, scales independently
Speech-to-Text	Self-hosted: Whisper large-v3-turbo (Russian/English, language detection) + Whisper fine-tuned on KSC2 (Kazakh); optional cloud fallback	Voice stays on our server; Kazakh quality
LLM	Cloud Flash/mini-class LLM with tool calling; local Qwen 3 / KazLLM (Ollama) for on-premise mode and fallback	Speed and reliability; private mode preserved
Database	PostgreSQL 16 + pg_trgm (+ pgvector)	Relational model, task search, JSONB
Cache, queues, pub/sub	Redis	AI queues, WebSocket across instances, idempotency
Voice files	MinIO (S3), encryption, 30-day retention	Privacy, portable to the cloud
API	REST (OpenAPI) + WebSocket	CRUD and a real-time board
Auth	JWT + refresh, Argon2, RBAC	Workspace and project permissions
Notifications	WebSocket + email, periodic reminders	Changes and deadlines
Infrastructure	Docker Compose (frontend, api, worker, postgres, redis, minio, ollama)	One-command startup

Layer	Technology	Key reason
Testing	pytest, httpx, Playwright (e2e), AI evaluation set (30+ scenarios), jiwer (WER)	Tests both the AI and system operations
CI and process	GitLab CI (linters, tests, build), YouTrack	Development trail

12 Validating the Choice

What is checked	Metric	Data set
STT	WER / CER per language, separately for auto and hint modes; latency	20+ recordings (kk/ru/en/mixed), `stt_benchmark.py`
Language detection	Kazakh/Russian/English detection accuracy (auto mode); routing of mixed recordings	Same recordings
Intent recognition	Accuracy over the eight actions and a “not a command” class	30+ scenarios from the Test Cases
Field extraction	Per-field accuracy (title, assignee, deadline, priority)	Same scenarios
Invented data	Share of values not present in the transcript	Same scenarios
Speed	Time from end of recording to preview (target: under 5 s)	Same scenarios
LLM comparison	Cloud and local models on the same set	Confirms Section 5

Results are stored in `benchmarks/results/` and summarised in this document.

13 Risks

Risk	Mitigation
Low STT accuracy on mixed speech	Manual language selection; transcript can be edited before processing
Cloud LLM unavailable	Fallback to the local model; manual task management always works
Wrong assignee or task match	Deterministic lookup and confirmation; clarification when several matches are found
No GPU for self-hosted STT	Smaller models and int8 on CPU; cloud STT fallback
Voice data leak	Private storage, encryption, retention limit, presigned URL access

14 References

[1] Study of STT/TTS for the Kazakh language (Whisper, GPT-4 Transcribe, ElevenLabs, Soyle). https://api.crossref.org/works/10.3390%2FINFO16100879

[2] Whisper fine-tuned on KSC2: https://huggingface.co/akuzdeuov/whisper-base.kk, https://huggingface.co/abilmansplus/whisper-turbo-ksc2

[3] Whisper large-v3 for Kazakh (tulpar). https://huggingface.co/olzhasAl/whisper-large-v3-tulpar

[4] faster-whisper large-v3-turbo. https://huggingface.co/deepdml/faster-whisper-large-v3-turbo-ct2

[5] ISSAI Soyle. https://github.com/IS2AI/Soyle

[6] ISSAI KAZ-LLM. https://issai.nu.edu.kz/kazllm/

[7] ElevenLabs Scribe for Kazakh. https://elevenlabs.io/speech-to-text/kazakh

[8] ElevenLabs API pricing. https://developer.puter.com/tutorials/elevenlabs-api-pricing/

[9] OpenAI API pricing. https://developers.openai.com/api/docs/pricing

[10] Gemini Flash: structured output, tool calling, pricing. https://docs.apiyi.com/en/api-capabilities/gemini-3-6-flash/overview.md